

**SIMATS ENGINEERING**  
**CO3 - ASSESSMENT TOOL 3**  
**Comparative Analysis**

**COURSE NAME: DEEP LEARNING**

**COURSE CODE: MLA0408**

**COURSE OUTCOME COVERED**

**CO3:** Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

**QUESTIONS:**

**Total Marks: 25**

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

#### Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is **considering AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

#### Q5. Regularized CNN vs Non-Regularized CNN

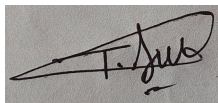
An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization** and a **CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

#### Code of Conduct:

I T. Narasimha (192425233) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink, appearing to read 'T. Narasimha', is written over a light gray rectangular background.

### RUBRICS FOR CALCULATION

| Criteria                        | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                             | Level 2 – Developing                    | Level 1 – Beginning                           |
|---------------------------------|-----------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------|-----------------------------------------------|
| Selection of Studies/Parameters | 20%       | Selects highly relevant and diverse studies with well-defined comparison parameters           | Selects relevant studies with minor gaps in parameter definition | Limited or partially relevant selection | Poor or inappropriate selection of studies    |
| Comparison & Differentiation    | 25%       | Clearly compares multiple aspects (methods, results, limitations) with strong differentiation | Good comparison with minor gaps                                  | Basic comparison with limited depth     | No clear comparison; mostly descriptive       |
| Critical Evaluation             | 25%       | Critically evaluates strengths, weaknesses, and implications with strong justification        | Provides evaluation with some justification                      | Limited evaluation; lacks depth         | No critical evaluation provided               |
| Synthesis & Insight Generation  | 20%       | Generates meaningful insights and conclusions from comparisons                                | Some insights derived from comparison                            | Limited or unclear insights             | No insights generated                         |
| Clarity                         | 10%       | Information is clearly presented (tables/figures) with logical structure                      | Generally clear with minor issues                                | Presentation lacks clarity or structure | Poor presentation and difficult to understand |

## ANSWERS

### Q1. CNN vs Fully Connected Neural Network

#### Comparative Analysis Table

| Parameter                    | Fully Connected Neural Network                                                                              | Convolutional Neural Network (CNN)                                                                                |
|------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Architectural Design         | Flat, fully-connected layers; every neuron connects to every neuron in the next layer                       | Layered design with convolution, pooling, and FC layers arranged hierarchically                                   |
| Motivation                   | Designed for generic, non-spatial data (tabular inputs)                                                     | Designed specifically to exploit spatial structure present in images                                              |
| Spatial Feature Extraction   | Image must be flattened into a 1-D vector, destroying spatial relationships between pixels                  | Convolutional filters slide over the 2-D image, preserving spatial relationships between neighbouring pixels      |
| Number of Parameters         | Extremely high — a 256x256 X-ray image flattened gives 65,536 inputs, each connected to every hidden neuron | Much lower — a small filter (e.g., 3x3) is shared across the whole image, drastically cutting parameter count     |
| Computational Complexity     | High memory and compute cost; scales poorly with image resolution                                           | Efficient — local receptive fields and shared weights reduce FLOPs and memory usage                               |
| Ability to Handle Image Data | Poor — loses edges, textures, and shape cues; prone to overfitting on high-dimensional pixel data           | Strong — learns hierarchical features (edges → textures → organ structures) directly suited to diagnostic imaging |

#### Justification / Analysis

- Convolutional layers apply small learnable filters across local regions of the X-ray image, detecting patterns such as edges, bone boundaries, and tissue density variations while preserving spatial context.
- Filters act as feature detectors; early filters capture low-level cues (edges, contrast) while later filters combine them into disease-relevant patterns (nodules, fractures, opacities).
- Pooling layers (max/average) down-sample feature maps, reducing dimensionality and computational load while making the learned features robust to small translations and noise in the scan.
- Because parameters are shared across spatial locations, CNNs require far fewer weights than a fully connected network of comparable depth, lowering overfitting risk on limited medical-imaging datasets.
- Given these advantages — reduced parameters, preserved spatial context, and translation robustness — a CNN is clearly the more suitable architecture for X-ray classification compared to a plain fully connected network.

## Q2. Different CNN Layers and Filters

Comparative Analysis Table

| Layer / Filter Type           | Role in Defect Detection                                                                                           | Key Characteristics                                                                                        |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Convolutional Layer           | Extracts local visual features such as edges, cracks, scratches, and texture irregularities on the product surface | Uses learnable filters (kernels); preserves spatial information; low parameter count due to weight sharing |
| Pooling Layer                 | Reduces the spatial size of feature maps while retaining the most important defect signals                         | Down-samples via max/average pooling; adds translation invariance; lowers computation                      |
| Activation Layer (e.g., ReLU) | Introduces non-linearity so the network can model complex defect patterns beyond simple linear boundaries          | Applied element-wise; computationally cheap; mitigates vanishing gradients compared to sigmoid/tanh        |
| Fully Connected Layer         | Combines all extracted high-level features to make the final defect/no-defect classification decision              | Dense connections; high parameter count; typically placed at the end of the network                        |
| Early-Layer Filters           | Detect low-level, generic features: edges, corners, colour gradients                                               | Small receptive field; broadly reusable across different defect types                                      |
| Deeper-Layer Filters          | Detect high-level, defect-specific features: shape of a crack, pattern of a dent, texture anomaly                  | Large receptive field; combine low-level features into semantically meaningful defect signatures           |

### Justification / Analysis

- Convolutional layers scan the product image locally, capturing fine-grained surface irregularities without losing their spatial location — essential for pinpointing where a defect occurs.
- Pooling layers compress this information, keeping the network efficient enough for real-time inspection on a production line while still retaining the strongest defect indicators.
- Activation layers allow the network to learn non-linear decision boundaries, letting it distinguish subtle real defects from harmless surface variations (lighting, reflections).
- Fully connected layers integrate the pooled, activated features into a single defect classification score, acting as the decision-making stage of the pipeline.
- Early filters build a generic vocabulary of edges and textures, while deeper filters recombine that vocabulary into specific, learned defect signatures — together this hierarchy lets the system move from raw pixels to an accurate, actionable defect classification.

### Q3. Parameter Sharing vs Independent Weights

#### Comparative Analysis Table

| Parameter                      | Independent Weights (Fully Connected Network)                                                                             | Parameter Sharing (CNN)                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Number of Parameters           | Very high — a separate weight for every input-neuron connection                                                           | Very low — the same small filter (e.g., 5x5) is reused across the entire image                                      |
| Memory Requirements            | Large memory footprint; impractical for thousands of high-resolution frames                                               | Compact memory footprint; feasible for continuous, high-volume video/image streams                                  |
| Computational Complexity       | Grows steeply with image resolution and network depth                                                                     | Grows much more slowly, since the same filter weights are re-applied via sliding convolution                        |
| Feature Detection              | Learns a unique, position-specific detector for each location — cannot generalize a pattern seen in one region to another | Learns one generic detector (e.g., edge/person-shape detector) that applies everywhere in the frame                 |
| Translation-Related Invariance | None — the network reacts differently if the same object appears in a different location in the frame                     | Built-in — the same filter fires consistently regardless of where the object appears, giving translation invariance |

#### Justification / Analysis

- By reusing the same filter weights at every spatial location, CNNs cut the parameter count by orders of magnitude compared to a fully connected network processing the same high-resolution frames.
- This directly reduces memory and compute requirements, allowing the surveillance system to process thousands of frames per hour on affordable hardware.
- Because a single filter learns a general pattern (e.g., a person's silhouette or a moving vehicle edge), it can detect that pattern anywhere in the frame — this is the source of translation invariance.
- Independent weights, by contrast, force the network to relearn the same pattern separately for every pixel location, wasting capacity and requiring far more training data to generalize.
- For large-scale, real-time image processing such as surveillance, parameter sharing is essential — it makes the system computationally feasible while improving generalization across camera views and object positions.

#### Q4. AlexNet vs ResNet

##### Comparative Analysis Table

| Parameter                     | AlexNet                                                                             | ResNet                                                                                                          |
|-------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Network Depth                 | Shallow — 8 layers (5 convolutional + 3 fully connected)                            | Very deep — variants from 18 to 152+ layers                                                                     |
| Layer Organization            | Sequential stack of conv, pooling, and FC layers                                    | Organized into residual blocks with skip/shortcut connections                                                   |
| Convolutional Filters         | Large filters in early layers (11x11, 5x5)                                          | Small, uniform filters (mostly 3x3) stacked in deep residual blocks                                             |
| Parameter Requirements        | ~60 million parameters, concentrated in FC layers                                   | Fewer parameters relative to depth due to smaller filters and global average pooling instead of large FC layers |
| Feature Extraction Capability | Good for moderately complex tasks; limited depth restricts hierarchical abstraction | Excellent — extreme depth captures highly abstract, hierarchical features                                       |
| Training Difficulty           | Relatively easier to train given shallower depth                                    | Would be very hard to train at such depth without residual connections                                          |
| Vanishing-Gradient Problem    | Present and worsens if depth is increased naively                                   | Solved via identity skip connections that let gradients flow directly through the network                       |
| Computational Cost            | Lower total depth but heavy FC layers increase cost there                           | Higher raw depth, but efficient design keeps overall cost manageable relative to accuracy gained                |
| Classification Performance    | Strong for its era (ILSVRC 2012 winner) but surpassed by deeper models              | State-of-the-art accuracy on large, complex, fine-grained classification tasks                                  |

##### Justification / Analysis

- AlexNet's shallow depth limits how many levels of abstraction it can learn, making it less suited to complex recognition tasks with many fine-grained classes.
- ResNet's residual (skip) connections allow the network to be trained much deeper without suffering from vanishing gradients, since gradients can flow directly through identity shortcuts during backpropagation.
- This lets ResNet learn richer, more hierarchical feature representations, which translates into higher accuracy on complex, large-scale image recognition benchmarks.

## Q5. Regularized CNN vs Non-Regularized CNN

### Comparative Analysis Table

| Parameter                  | CNN without Regularization                                                             | CNN with Dropout, Data Augmentation & Batch Normalization                                                  |
|----------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Overfitting                | High — model memorizes training leaf images, including noise and background artifacts  | Significantly reduced — dropout and augmentation force the network to learn generalizable disease features |
| Generalization             | Poor performance on unseen leaf images from new plants/conditions                      | Strong performance on unseen images due to exposure to varied, augmented samples                           |
| Training Stability         | Can be unstable; internal covariate shift slows and destabilizes convergence           | More stable — batch normalization normalizes layer inputs, smoothing the loss landscape                    |
| Computational Requirements | Lower per-epoch cost but often needs more epochs / careful tuning to avoid overfitting | Slightly higher per-epoch cost, but converges more reliably and reduces wasted training cycles             |
| Model Performance          | High training accuracy, low real-world/deployment accuracy                             | Balanced training and validation accuracy; better real-world reliability                                   |

### Justification / Analysis

- Dropout randomly deactivates neurons during training, preventing the network from relying too heavily on any single feature and reducing memorization of training-specific leaf artifacts.
- Data augmentation (rotation, flipping, brightness/contrast changes) simulates the natural variation seen in real field images — different lighting, leaf orientation, and camera angles — improving robustness.
- Batch normalization stabilizes and speeds up training by normalizing intermediate activations, which also has a mild regularizing effect that further curbs overfitting.
- Given the initial model's high training accuracy but poor generalization, applying all three techniques together — dropout, data augmentation, and batch normalization — is the most appropriate approach for a reliable, field-deployable plant disease classifier.